

Background: Classical Compilers, Quantum Circuit Compilers, FTQC Compilers

Michael Ferguson

OpenQSE Day, July 24 2026

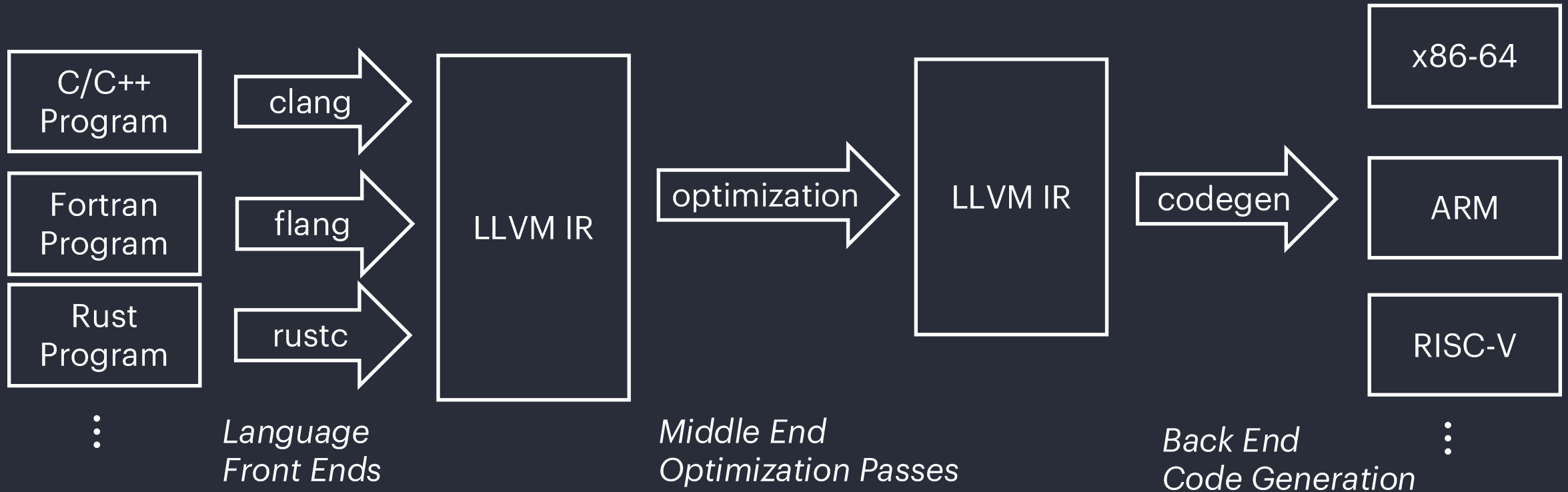
Classical Compilers

What is the typical architecture of a classical compiler?

LLVM



llvm.org



“IR” = “Intermediate Representation” = a way to represent programs across many languages
LLVM IR models a general CPU instruction set; close enough to common patterns to optimize
LLVM IR can also represent GPU code

Multi-Level Intermediate Representation

MLIR is a framework for specifying IRs, combining IRs, and writing passes that transform IRs

- An IR specified in MLIR is called a “Dialect”
- “Lowering” is a common thing a pass might do; it means translating from one dialect to another (usually lower-level) one

An MLIR representation of a program can include features of many Dialects at once

MLIR was developed primarily by the Machine Learning community (TensorFlow)

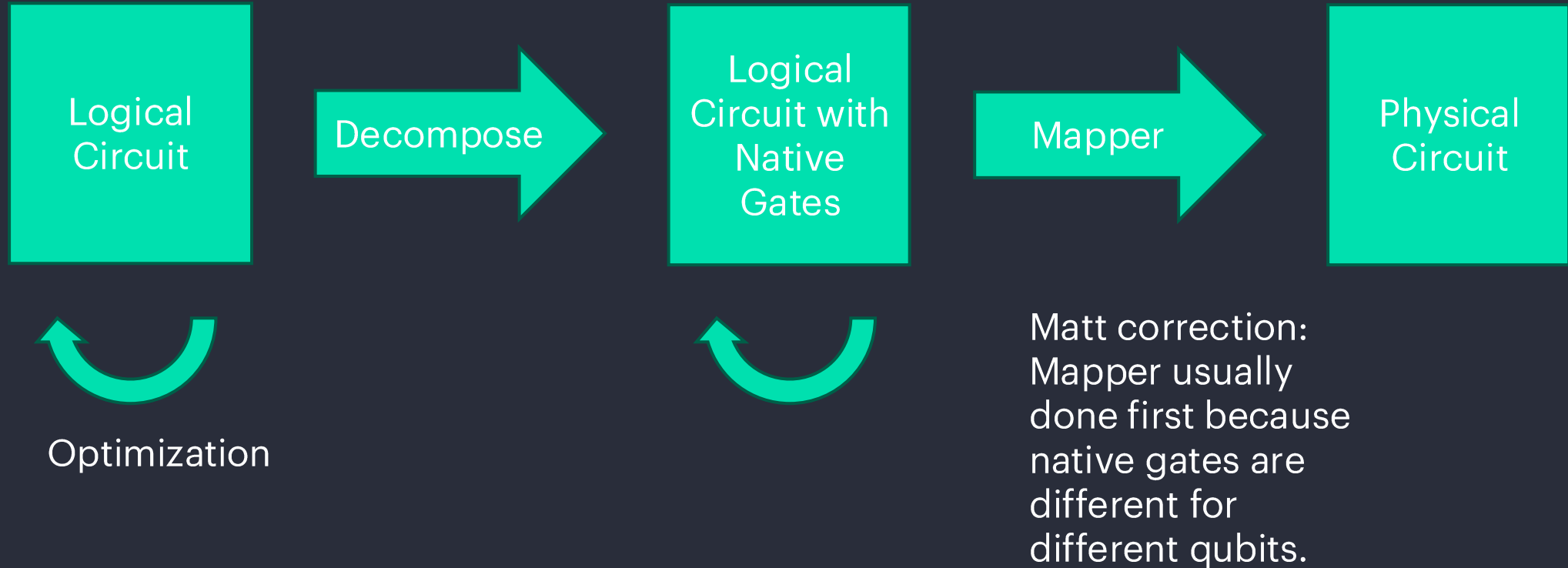
It is also used by PennyLane (with the Catalyst dialect) and CUDA-Q (with the Quake dialect)

The ability to combine dialects makes it a great match for hybrid classical + quantum programs

Quantum Circuit Compilers

What are quantum circuit compiler stages? How are they different in NISQ and FTQC?

NISQ Compiler Sketch

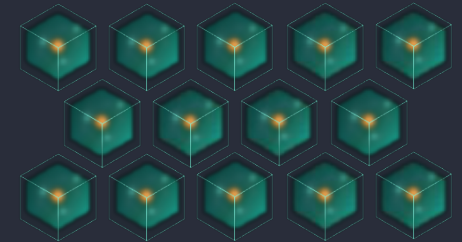


Fault-tolerant quantum computation (FTQC) vs Noisy intermediate-scale quantum (NISQ)

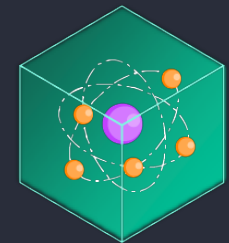
- Noise in current hardware limits the largest quantum computations that one can run to $\sim \log(n)$ -depth (where n is the number of qubits).
- Quantum Error Correction (QEC) reduces errors by using more qubits to encode information + encoding operations.
- New challenges: e.g., no universal fault-tolerant gates [Eastin–Knill theorem]
- Rotation gates need to be approximated with long (~ 300 gates) sequences of other gates, and T gates need to be provided separately (e.g. with magic state distillation and injection)
- Key distinction from NISQ: minimizing T gates as opposed to CNOTs.
- Real-time decoding requires significant classical compute. We see the need for additional classical compute.
- Compiling quantum algorithms will be different: approximate rotation gates, schedule magic state distillation, lattice surgeries, etc.

In summary, FTQC brings new challenges: scaling up qubits, different compilation stack, additional classical compute, etc.

Quantum Error Correction

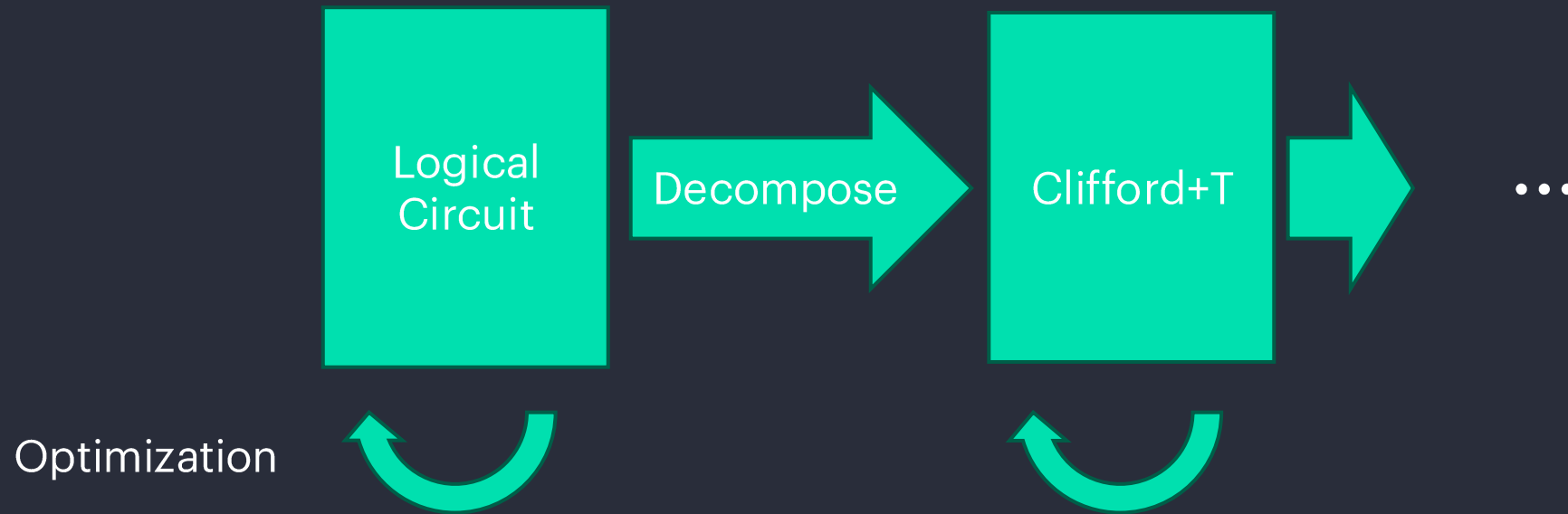


Many noisy physical qubits



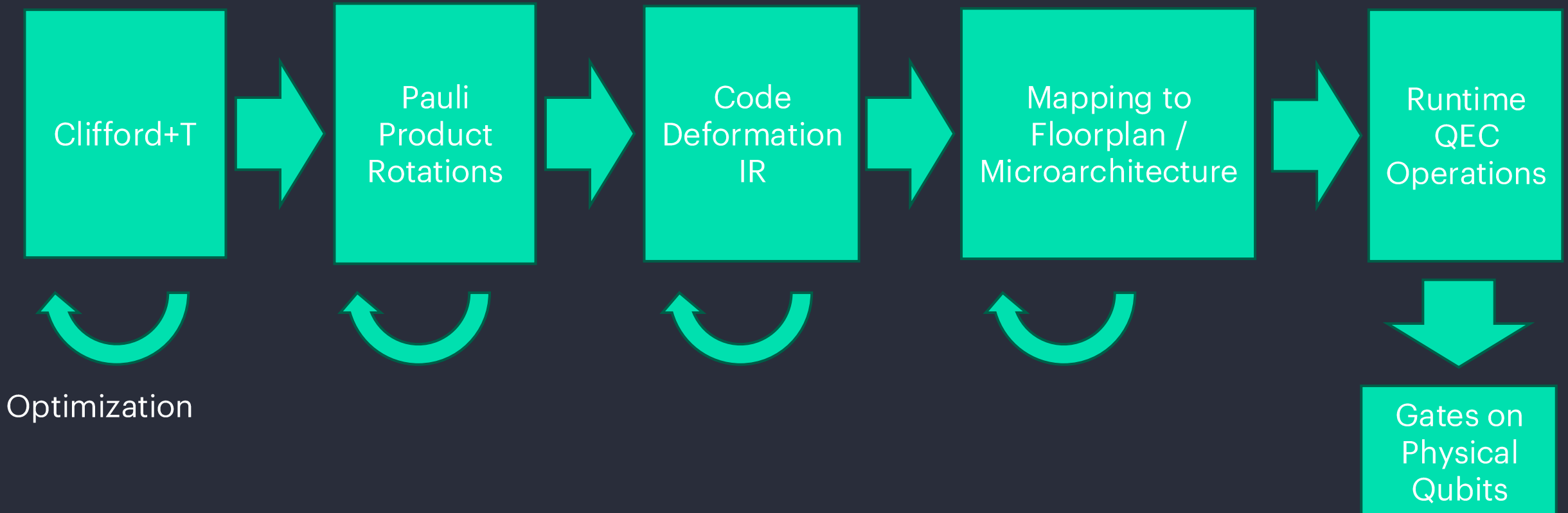
Logical qubit

FTQC Compiler Sketch: Starting Point



FTQC Compiler Sketch: Stationary Qubits

“micro ops”
probably in HW



Optimization Technique: PPR

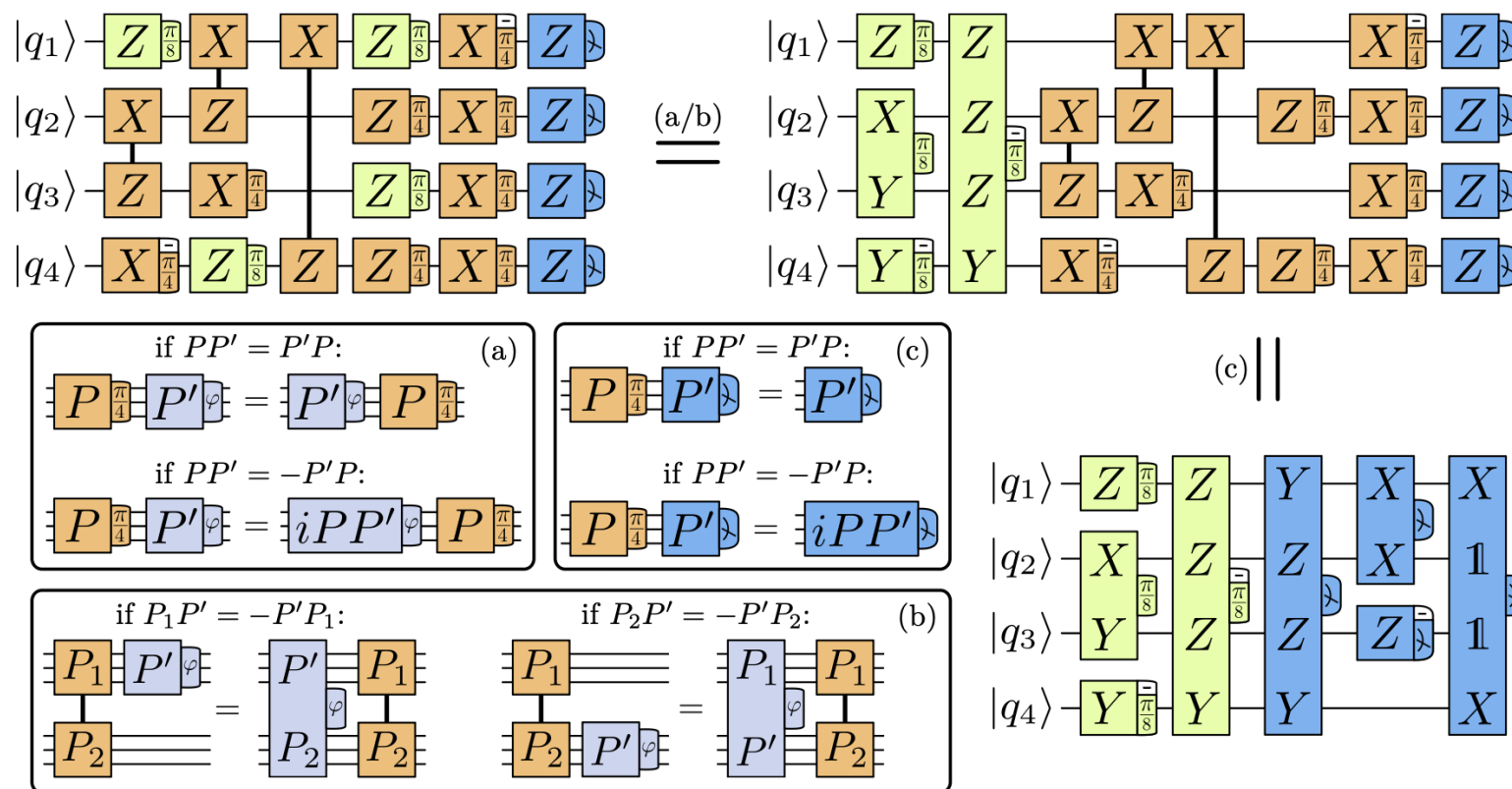


Figure 4: A generic circuit consists of $\pi/4$ rotations (orange), $\pi/8$ rotations (green) and measurements (blue). The Pauli product in each box specifies the axis of rotation or the basis of measurement. If the Pauli operator is $-P$ instead of P , a minus sign is found in the corner of the box, such that, e.g., $Z_{-\pi/4}$ corresponds to an $S^\dagger$ gate. Using the commutation rules in (a/b), all Clifford gates can be moved to the end of the circuit. Using (c), the Clifford gates can be absorbed by the final measurements.

Q: Is PPR capable of being generalized to hybrid quantum-classical programs?

How to handle mid-circuit measurements with PPR?

Conditional gates?

Loops or other control flow?

FTQC Floorplan

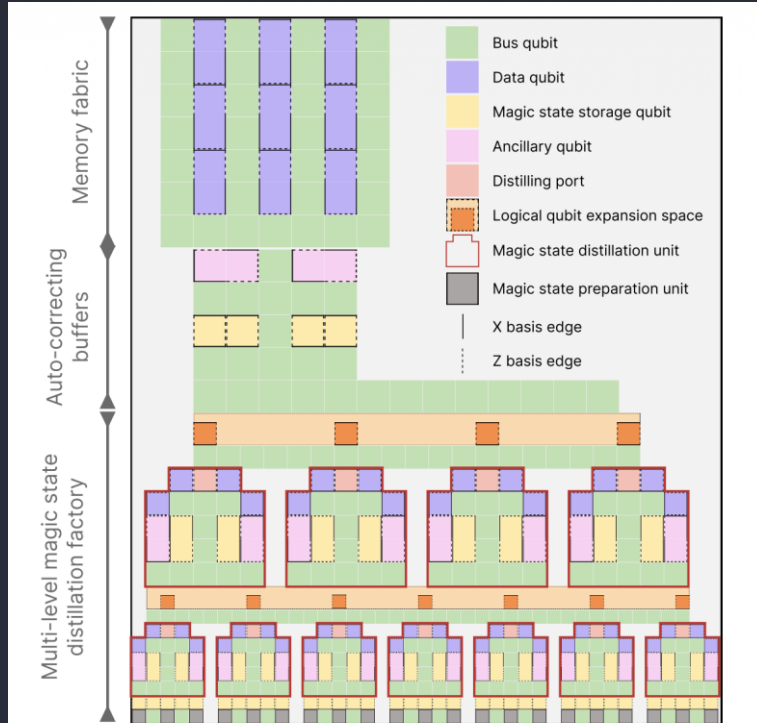


Figure 16: Example of a logical layout of a modular fault-tolerant quantum architecture of the sort designed by TopQAD [78, 79, 177]. In this example, two distillation levels are used, composed of four and two distillation units each from lowest to highest.

[Mohseni et al., How to Build a Quantum Supercomputer: Scaling from Hundreds to Millions of Qubits](#)

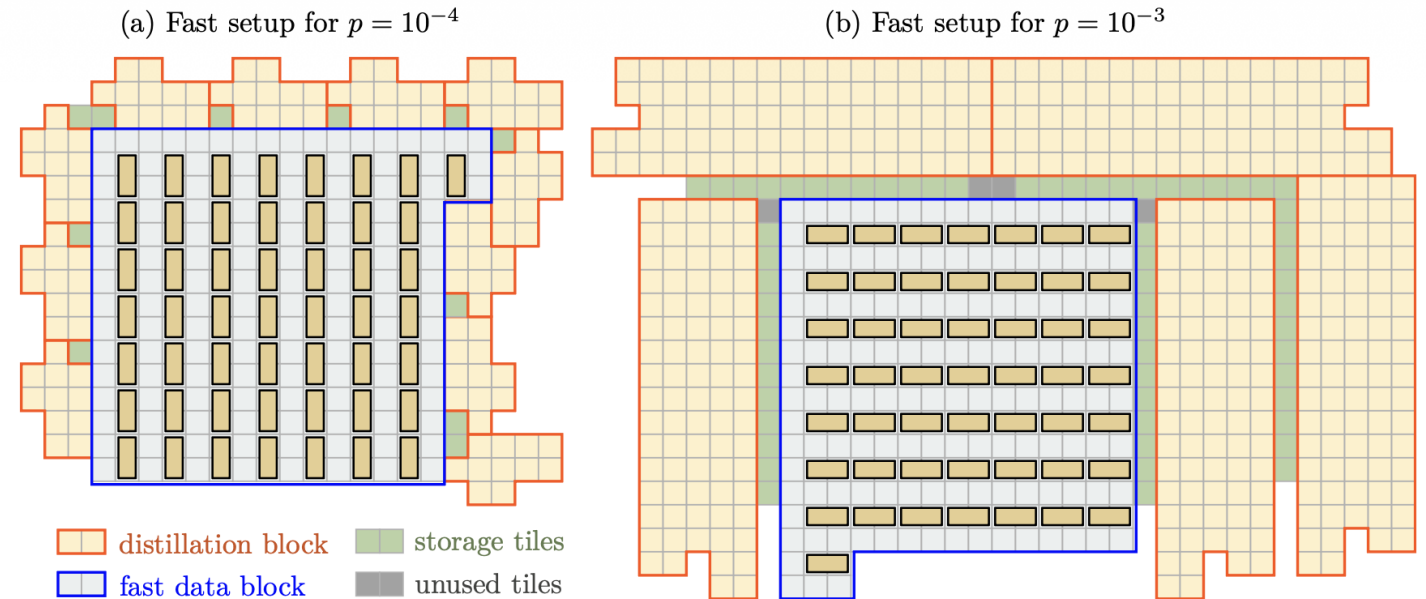
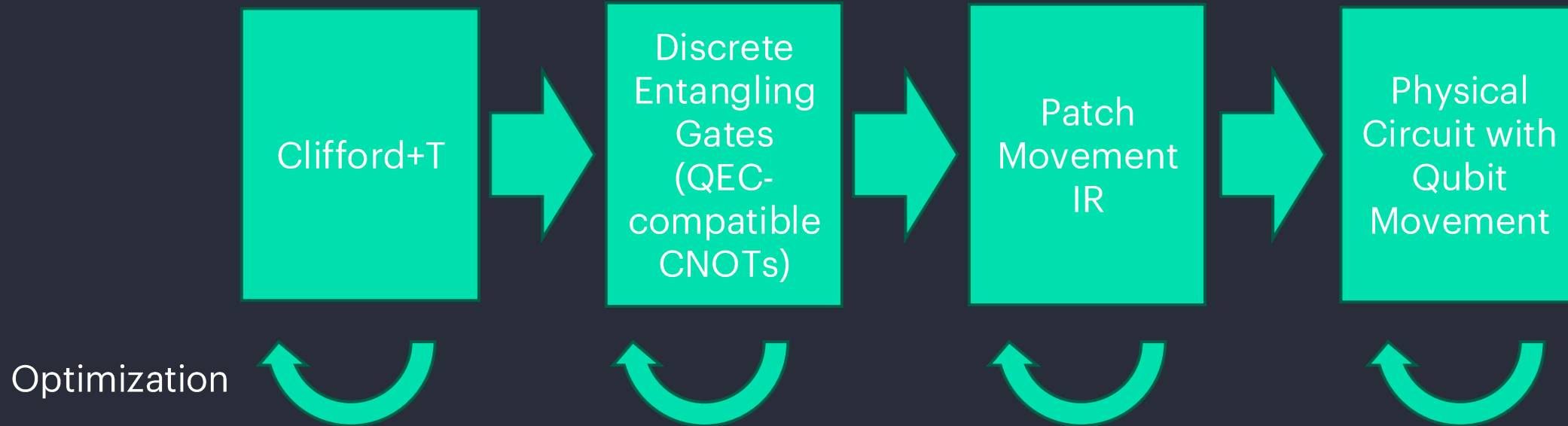


Figure 23: Fast setups using fast data blocks and 11 15-to-1 distillation blocks for $p = 10^{-4}$ or 5 116-to-12 distillation block for $p = 10^{-3}$.

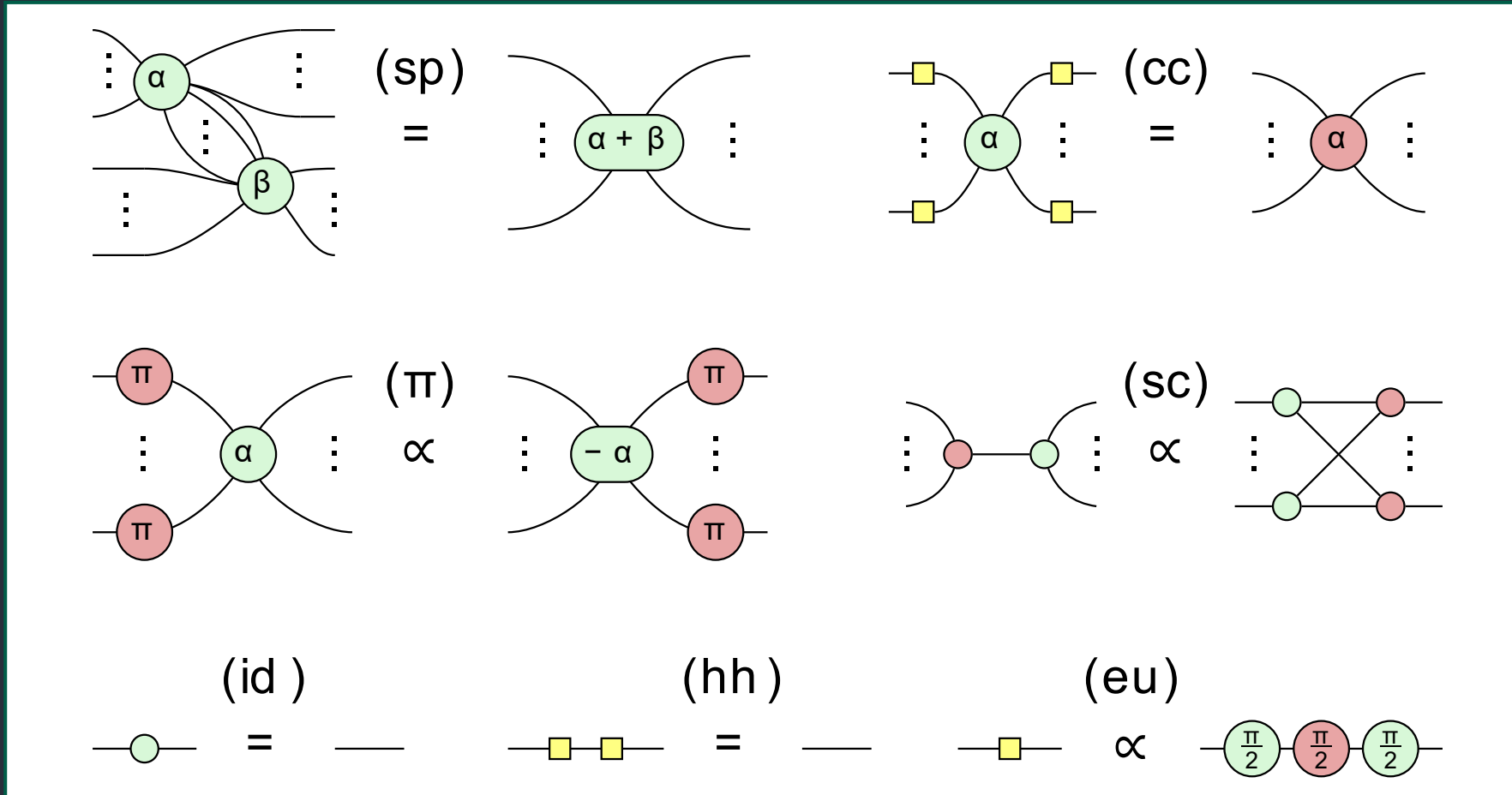
[Daniel Litinski, A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery](#)

FTQC Compiler Sketch: Moving Qubits



qLDPC codes represent multiple logical qubits per patch and moving qubits allow transversal CNOTs but not CNOTs within a patch

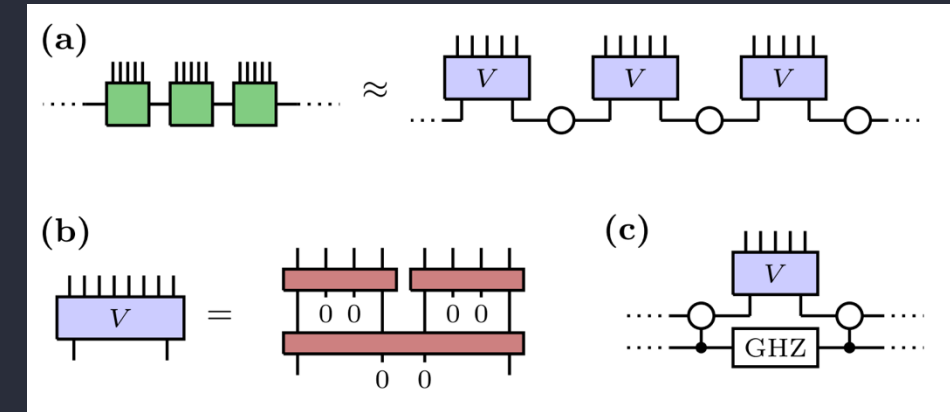
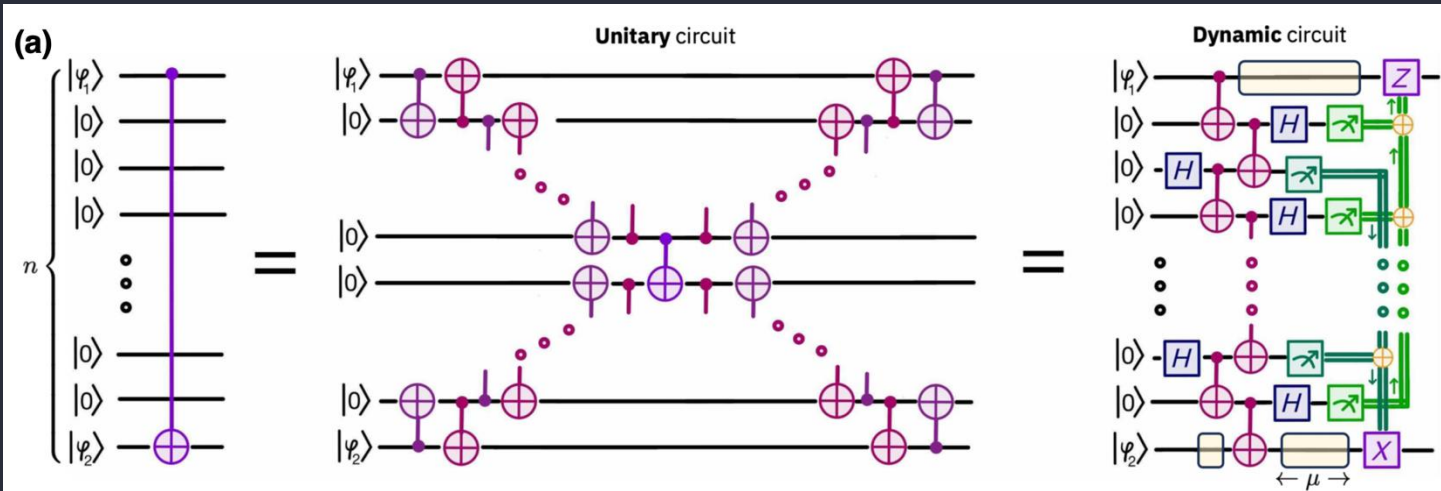
Optimization Technique: ZX Calculus



[Alex Kissinger and John van de Wetering, Picturing Quantum Software An Introduction to the ZX-Calculus and Quantum Compilation](#)

Mid-Circuit Measurements and Feed Forward

MCM: Dynamic circuits for accelerating quantum state prep

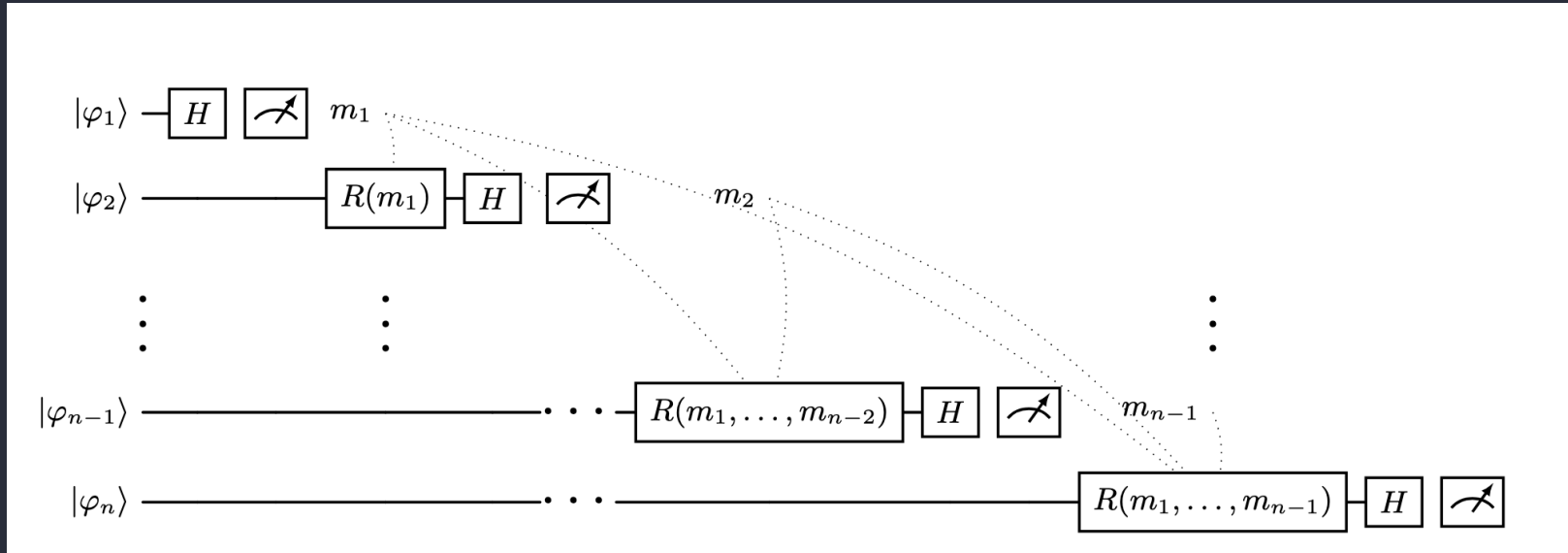


- Measurements and feedback can prepare arbitrary translation-invariant MPS (“tensor network states”) with an exponential speedup than purely unitary synthesis: from depth $O[\log(N/\epsilon)]$ to depth $O[\log \log(N/\epsilon)]$.
- Such MPS states provide an excellent approximation to the GS.
- Note: this measurement-and-feedback circuit is at the *logical* level.

[1] Efficient Long-Range Entanglement Using Dynamic Circuits [PRX Quantum 5, 030339 (2024)]

[2] Preparation of Matrix Product States with Log-Depth Quantum Circuits [Phys. Rev. Lett. 132, 040404 (2024)]

QFT+M: Semi-Classical Procedure on FTQC



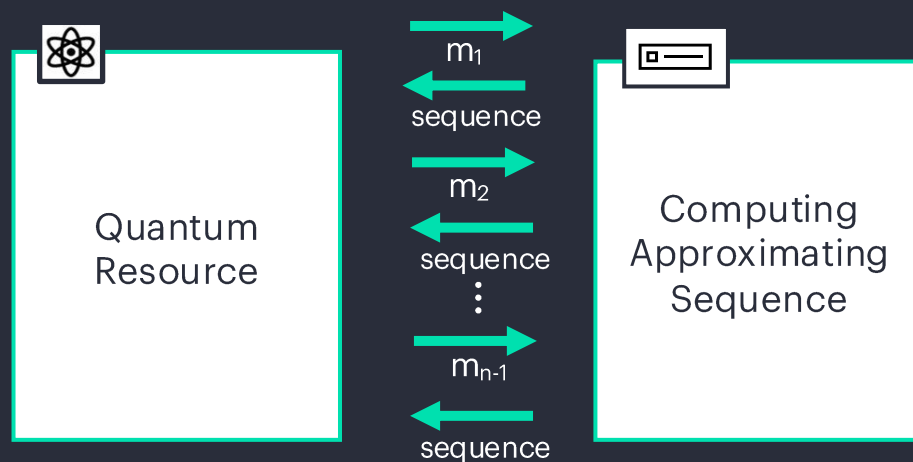
How to extend quantum SDKs and circuit submitted to QPUs to support this pattern & other mid-circuit HPC-QC?

The angles of the gates depend on the measurement results.

FTQC systems approximate rotations with approximating gate sequences

Need to compute an approximating gate sequence mid-circuit!

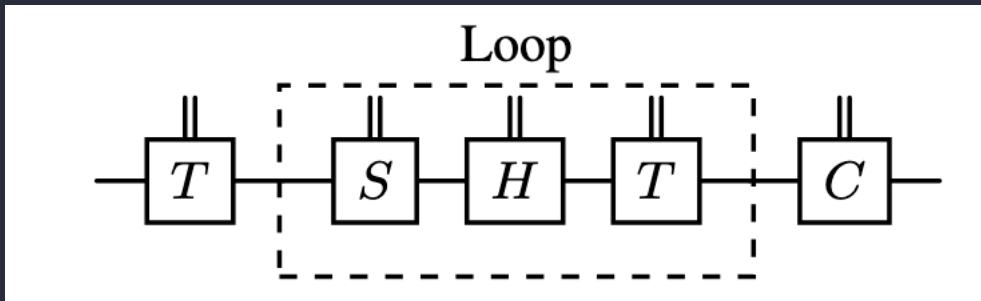
This cannot be done in control electronics. Therefore, it is not possible within the initial system design.



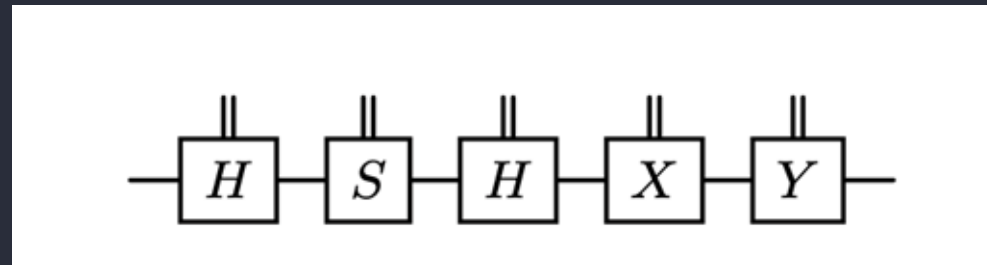
GECCOs: Gates with Externally-Controlled Conditional Operations

We can adjust the system design to support this pattern:

- Control electronics support classically conditional gates where the condition comes from **outside the quantum resource** & mid-circuit measurements are sent externally as well
- We use the Matsumoto-Amano normal form for any single-qubit unitary: $(T|\epsilon)(HT|SHT)^*C$



Any single-qubit unitary can be represented as a loop with classical controls to indicate which unitary



Any single-qubit Clifford gate can be represented similarly

For example:

SHTSHTHTHTHTSHTHTSHTSHTSHX

Can be represented within 12 loops using 0 bits to skip unused operations:

01111110110110110111110111111100000001110

TSHTSHTSHTSHTSHTSHTSHTSHTSHTSHTSHTSHXY